

现有案例匹配流程分析

1. 文档目的

本文基于当前代码实现，整理现有案例匹配流程的：

1. 目标
2. 当前方法与执行流程
3. 主要问题
4. 改进方案
5. 对接入外部 API 的处理建议

2. 现有案例匹配流程的目标

案例匹配并不只是“查相似问句”，而是承担了几层目标。

2.1 复用已验证的逻辑

案例中会保存示例问题，计算表达式，可变参数等信息，把已经生成并验证过的 SQL 或 Python 逻辑沉淀下来，在后续遇到相似问题时直接复用，而不是每次都重新生成。

2.2 将稳定逻辑与可变参数分离

当前案例结构把逻辑计算模板与可变参数定义分开保存，目标是把问题如何计算的逻辑生成过程固定下来，只把“时间、区域、对象、TopN”等变化部分作为参数提取和替换。

2.3 让匹配路径成为快速路径

匹配速度的计算量更小，案例路径比重新生成更快、更稳定。

2.4 分阶段目标

1. 变量变化：参数发生变化
2. 语序变化：词组的语序调整
3. 关键词变化（表达多样化）：同一语义的不同表达形式，需要处理同义表达处理，可能需要业务知识
4. 口语化：涉及缩写、省略、非正式表达和隐式语义补全

分类	输入示例	目标案例表达
顺序变	销售总金额在2023年是多少？	2023年销售订单总金额是多少？
关键词变	2023年的销售总额是多少？	2023年销售订单总金额是多少？
口语化	帮我查一下23年一共卖了多少钱？	2023年销售订单总金额是多少？
关键词变	2025年12月订单量环比11月变化？	2025年12月订单量环比11月变化多少
关键词变	25年12月订单量环比情况如何？	2025年12月订单量环比11月变化多少
顺序变	电子产品2024年销量占比多少？	2024年电子产品的销量占总销量多少比例？
关键词变	24年电子产品销售数量占比如何？	2024年电子产品的销量占总销量多少比例？
关键词变	2024年电子产品的销量占总销量多少？	2024年电子产品的销量占总销量多少比例？
口语化	25年仓库进了多少货，出了多少货？	2025年库存入库总量、出库总量是多少
关键词变	统计2025年的入库总数和出库总数	2025年库存入库总量、出库总量是多少
顺序变	24年VIP客户平均一单花多少钱？	VIP客户2024年平均客单价是多少
口语化	VIP客户24年平均每单花多少钱	VIP客户2024年平均客单价是多少
口语化	把23年每个月的销售额列出来	2023年1-12月每月订单金额分别是多少
口语化	我要2023年1月到12月的月度订单金额	2023年1-12月每月订单金额分别是多少
关键词变	按月统计23年的订单金额	2023年1-12月每月订单金额分别是多少

最终目标是“完全不同结构但语义等价”，为了达到这个目标，需要将问题拆分为具体的词组成分，并在每个词组中将同构改写稳定下来。与我们的目标对应的，每个阶段需要解决的关键问题包括：

- 能够从问题中识别对应同一个参数的变量
- 能把不同语序的问题归并到同一个案例
- 能够识别问题中起关键作用的表达词组（环比、占比）

- 能把“销售总金额 / 卖了多少钱”这类表达归并到同一个指标语义

第一阶段：变量变化

这一阶段解决的是：

- 问题的主干逻辑不变
- 只是时间、地区、客户、项目、TopN 之类参数发生变化

典型问题包括：

- 2024年四公司产值最高的项目
- 2025年四公司产值最高的项目

这一阶段的主要问题不是语义理解，而是“如何把变化部分稳定地从问题中抽出来，并与案例模板中的参数对应起来”。

主要问题：

- 参数和固定逻辑没有被明确分开
- 不同参数值会影响召回文本，导致同类问题不稳定
- 时间、组织、地域等参数虽然看起来简单，但如果没有标准化，后续仍然会漂移

解决手段：

- 对案例先建立稳定的规范表达，明确哪些部分是固定逻辑，哪些部分是参数槽位
- 基于数据字段对参数建立清晰的槽位定义
- 在问题解析阶段优先做参数抽取，对抽出的参数做标准化
- 匹配时优先比较固定逻辑骨架，不让参数值干扰案例归并

这一阶段完成后，系统至少要能做到：

- 同一案例模板在参数变化时仍稳定命中
- 命中后可以直接把参数带入执行，而不需要重新理解整个问题

第二阶段：语序变化

这一阶段解决的是：

- 词还是那些词
- 只是词组顺序变化了

典型问题包括：

- 销售总金额在2023年是多少？
- 2023年销售总金额是多少？

这一阶段的主要问题是，当前链路很容易把语序当作结构变化，从而影响角色生成和后续匹配。

主要问题：

- 当前匹配对表层表达顺序比较敏感
- 问题角色是逐案生成的，语序变化时容易生成出不同结构
- 如果过度依赖整句 embedding，相似度能看起来接近，但不够稳定，也不够可解释

解决手段：

- 在问题解析时先把句子拆成稳定的语义片段，而不是按原句顺序直接参与匹配
- 先识别出“时间”“指标”“主体”“限定条件”等稳定成分，再重组为统一的问题表达
- 对匹配过程做“顺序弱敏感”处理，也就是优先比较成分是否齐全、关系是否一致，而不是比较原句中谁在前谁在后
- 为案例侧也建立统一的规范表达，让问题和案例都落到同一套成分顺序上

这一阶段的重点不是让模型更聪明地“猜结构”，而是减少表层顺序对结构判断的干扰。

这一阶段完成后，系统要能做到：

- 同样的语义成分，即使换了顺序，也能落到同一个案例
- 匹配理由能解释成“结构相同，只是表述顺序不同”

第三阶段：关键词变化

这一阶段解决的是：

- 结构大体相同
- 但用了不同的词来表达同一语义

典型问题包括：

- 统计2025年的入库总数和出库总数
- 2025年库存入库总量、出库总量是多少

这一阶段开始真正涉及业务术语归一和表达多样化处理。

主要问题：

- 同一业务概念有多种说法
- 同一个词在不同业务空间中可能含义不同
- 有些词表面相近，但对应的真实口径并不一致
- 只靠通用同义词并不够，很多时候还需要业务知识

解决手段：

- 为每个案例补充足够的 `examples`，让案例本身覆盖更多真实问法
- 建立业务术语归一层，把常见同义表达先映射到规范说法
- 引入业务知识参与归一，例如：
 - 客单价 和 平均每单花多少钱 归到同一指标语义
 - 入库总数 和 入库总量 归到同一统计对象

这一阶段的重点是把“不同说法”先拉回到“同一个规范语义”，再做结构匹配。

这一阶段完成后，系统要能做到：

- 同义表达不再成为案例命中的主要障碍
- 召回阶段不再过度依赖 `case_name`
- 匹配链路开始真正利用 `examples` 和业务知识

第四阶段：口语化

这一阶段解决的是：

- 缩写
- 省略
- 非正式表达
- 隐式语义补全

典型问题包括：

- 帮我查一下23年一共卖了多少钱？
- 把23年每个月的销售额列出来

这一阶段最难，因为它不只是“换了词”，而是经常把完整问题压缩成不规范表达。

主要问题：

- 时间、对象、统计口径常常被缩写
- 主语和限定条件可能被省略
- 有些语义需要结合业务知识补全
- 口语表达中经常混入模糊词，例如“情况如何”“查一下”“列出来”

解决手段：

- 在统一入口中增加一层口语规范化，把明显缩写和口头表达先改写成规范问题
- 增加常见缩写和口语映射步骤，例如：
 - 23年 -> 2023年
 - 卖了多少钱 -> 销售总金额
 - 每单花多少钱 -> 平均客单价
- 对省略信息结合案例骨架、业务知识和 schema 约束做补全
- 对补全结果保留置信度，只有在置信度足够时才直接进入精确匹配
- **如果存在多个合理解释，就进入歧义状态，而不是盲目命中一个案例**

这一阶段的重点不是“让模型自由发挥补全”，而是“让补全受到案例骨架、业务知识和数据约束的共同限制”。

这一阶段完成后，系统要能做到：

- 常见缩写和口头表达可以稳定落回规范问题
- 隐式语义补全不再完全依赖模型主观猜测
- 口语问题也能进入和正式问题相同的匹配主链路

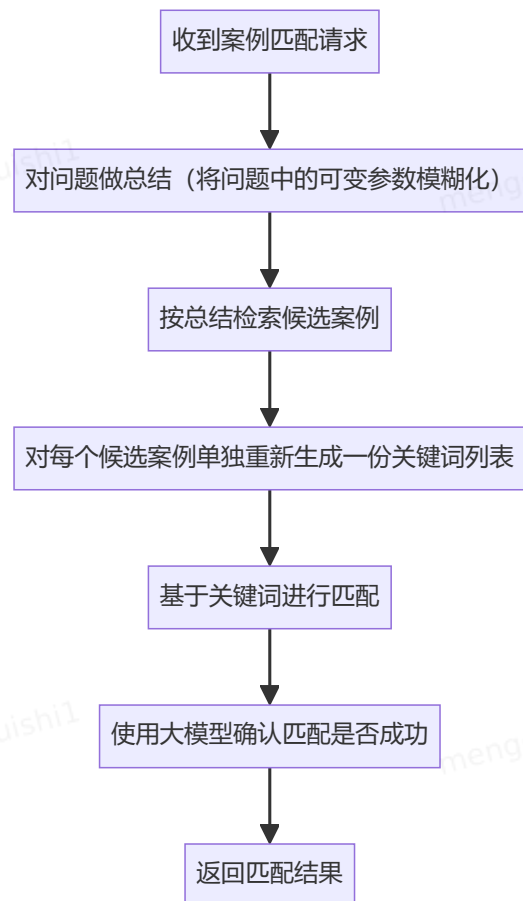
3. 当前方法与实现流程

3.2 当前主流程

当前真正执行匹配的是 `CaseManagementProcessor.match_text_with_cases`。整体流程如下：

文本绘图

```
1 flowchart TD
2     A[收到案例匹配请求] --> B[对问题
    做总结（将问题中的可变参数模糊化）]
3     B --> C[按总结检索候选案例]
4     C --> F[对每个候选案例单独重新生
    成一份关键词列表]
5     F --> G[基于关键词进行匹配]
6     G --> H[使用大模型确认匹配是否成
    功]
7     H --> I[返回匹配结果]
```



3.3 候选案例召回

当前先对原始问题执行一次 `summary()`，然后把摘要结果作为 `retrievalEmbedding` 传给远程案例检索接口 `_get_existing_cases(...)`。

请求参数包括：

- `businessSpaceId`
- `datasourceId`
- `modelIds`
- `retrievalEmbedding`
- `topK=10`
- `publishStatus=PUBLISHED`

这一步依赖外部案例服务返回候选案例，再将响应直接反序列化为 `CaseModel`。

3.4 候选案例上的逐案重解析

召回后，不是先为当前问题生成一份统一、稳定的问题结构表示，而是对每个候选案例都单独执行一次：

1. 用 `formatted_intent_info` 展平得到 `base_reference`
2. 取该候选案例的 `statement.template` 和 `parameter`

3. 调用 `generate_roles(match_question, base_reference, case_template, case_parameters, additional_knowledge)`

4. 得到一份“针对这个候选案例上下文生成出来的 question roles”

这意味着，同一个问题面对不同候选案例时，系统可能生成出不同的结构拆分结果。

3.5 案例相似度判断

逐案生成 roles 后，系统会调用

`CaseSimilarityMatcher.check_similarity_with_cases(...)`。

当前匹配逻辑大致分三层：

1. 收集问题文本、问题 roles、案例 `case_name`、案例 roles 的关键词
2. 调用 `TextEmbeddingManager` 获取 embedding
3. 先做整体语义筛选，再做角色级匹配，最后在角色匹配通过时再做 SQL 逻辑校验

当前几个关键点是：

- 整体语义筛选用的是“问题文本”和案例的 `case_name`
- 如果没有任何案例达到语义阈值，会保留相似度最高的一个案例继续比较
- 角色匹配通过后，还会再调用 LLM 做 `_check_sql_logic_match`
- `exact_match` 的成立条件是“角色匹配通过且 SQL 逻辑校验通过”

3.6 返回结果

当前返回结构是：

```
1  {
2    "match": true,
3    "exact_match": false,
4    "match_case_id": ["case_xxx"]
5  }
```

其中：

- `match` 表示是否存在相关案例
- `exact_match` 表示是否存在精确匹配案例
- `match_case_id` 优先返回精确匹配案例 ID；如果没有精确匹配，则返回模糊匹配案例 ID

4. 当前方法的主要问题

4.1 问题表示不稳定

这是当前方案最核心的问题。

系统没有先把自然语言问题沉淀成一份统一、稳定、可复用的问题表示，而是在每个候选案例上重新生成一份 roles。这样会导致：

- 同一句问题在不同候选案例上下文中被拆成不同结构
- 匹配结果依赖候选案例上下文，而不是问题本身
- 难以保证多次请求得到一致表示

这也是实践中“无法稳定把自然语言转换为相同问题表示”的根源之一。

4.2 整体语义筛选没有充分利用 examples

虽然案例模型里有 examples，但当前 CaseSimilarityMatcher 在做整体语义比较时，实际使用的是 case_name，而不是 examples 集合。

这会带来两个结果：

- 标题起得好不好，会强影响能否进入后续匹配
- 实际逻辑接近、但标题写法不同的案例，容易在前置筛选阶段被误伤

4.3 匹配与提参没有共享同一套稳定语义骨架

当前匹配依赖“逐案生成的 roles”，参数提取则在

extract_parameters_by_case(...) 中重新组合 roles + statement + parameters + additional_knowledge 去让 LLM 提参。

两者虽然都用到了 roles，但没有共享一套稳定、标准化的问题中间表示，因此：

- 匹配命中的依据和提参理解的依据并不完全一致
- 命中后提参仍然可能偏移
- 系统很难解释“为什么匹配对了，但参数又提错了”

5. 改进方案

5.1 建立稳定的案例表示

现在对案例的中间表示，是让大模型完成的，案例的问题和sql都只是作为参考，没有明确的基准，导致问题表示不稳定

案例的生成应该以数据库schema为准：

例如：

用户问题：

‘查询今年华东区域销售额最高的前10个项目’

之前：查询某年某区域销售额最高的前10个项目

现在：查询[stat_date][region]销售额最高的前10个项目

```sql

SELECT



```

project_name,
SUM(sales_amount) AS sales_amount_sum
FROM dws_project_sales_day
WHERE stat_date >= {start_date}
AND stat_date <= {end_date}
AND region_name = {region}
GROUP BY project_name
ORDER BY sales_amount_sum DESC
LIMIT {top_n}
...

```

| 问题原文  | 槽位    | schema                            |
|-------|-------|-----------------------------------|
| 今年    | 时间    | stat_date                         |
| 华东区域  | 华东    | region_name                       |
| 销售额最高 | 销售额最高 | ORDER BY<br>sales_amount_sum DESC |
| 前10个  | 前n个   | LIMIT {top_n}                     |
| 项目    | 项目    | project_name                      |

这个基准应该是：

- schema 中真实存在的表和字段
- 字段之间允许形成的聚合、过滤、排序和分组关系

因此，后续案例生成遵循下面的顺序：

1. 先根据 schema 确定可绑定的字段和槽位
2. 再从问题中摘取关键词，生成稳定的问题表示
3. 最后根据sql中的槽位将问题表示中的可变量进行标注

除了案例的整体逻辑骨架要稳定，案例中的可填充参数也必须被明确规定，不能继续只靠大模型在匹配时自由猜测。

这里的关键思想是：

- 槽位不是一个抽象名字
- 槽位必须明确绑定到 schema 中的字段
- 槽位能接受什么值，也必须有明确约束

也就是说，案例表示里应该写成：

- 变量对应哪个字段
- 这个字段的值域从哪里来
- 匹配成功后返回什么标准值

对于这类槽位，尤其是组织、区域、项目、产品等实体类槽位，推荐直接基于现有字段中的枚举值或维表值来定义可填充范围。

例如，一个案例里如果定义了组织槽位，就不能只说“这里可以填组织”，而应进一步规定：

```
1 select count(1) from project_table where project_name =
 {project_name}
```

这意味着：

- 这个槽位不是开放文本
- 它只能绑定到数据库中真实存在的组织值
- 用户问题中的词语，必须先和数据库中的可用值建立关联，才能被认定为该槽位的合法填充值

## 5.2 案例匹配过程的重构

### 第一步：参数识别和验证

第一步先解决“问题里哪些部分是变量，哪些部分是固定逻辑”的问题。

很多看上去不一样的问题，本质上只是参数变了，逻辑并没有变。如果这一步不先做，系统就会把大量本来属于同一案例的问题，拆成很多不稳定的匹配结果。

- 典型问题：
  - 时间、组织、区域、项目、TopN 这类内容经常和问题主干混在一起
  - 同一个词可能既像参数，又像业务描述，容易识别错位
  - 参数如果不先标准化，后面每一步都会继续放大偏差
- 解决方式：
  - 先把可变部分单独识别出来，不让它们直接参与主逻辑比较
  - 对识别出的参数做标准化，把缩写、别名和口头说法统一到稳定表达
  - 对组织、区域、项目等实体类参数增加校验，尽量确认它们能否落到真实业务对象上
- 示例：
  - 今年四公司产值最高的项目
  - 去年四公司产值最高的项目
  - 这两个问题的核心逻辑都可以先收敛为“查询某时间范围内四公司产值最高的项目”

- 两者的主要差别只在时间参数，一个对应“今年”，一个对应“去年”
- 如果 **四公司** 可能被误解为项目名或组织名，就需要先校验它更合理地落在哪个业务对象上，再进入后续匹配
- 阶段结果：
  - 问题里的变量被单独提取出来
  - 变量被尽量转换成可复用、可比较的稳定结果
  - 后续链路比较的是“主干逻辑 + 已确认参数”，而不是整句原文

## 第二步：词组归一化

第二步解决的是“问题表面怎么说”和“案例内部怎么表达”之间的差异。

参数确定之后，系统面对的下一个问题是：同一个意思，用户可以用很多不同的方式说出来。有时只是语序换了，有时是关键词换了。如果这一层不处理，系统就会把表面差异误判成结构差异。

- 典型问题：
  - 同一个问题换个语序就像换了一个结构
  - 同一个业务概念有多种叫法
  - 不同业务空间里，同一个词还可能有不同含义
- 解决方式：
  - 不再直接按原句顺序比较，而是先整理出时间、指标、主体、限定条件等核心成分
  - 把常见同义表达、业务别名、历史叫法逐步归到规范说法
  - 对一眼看不准的词先保留多种可能，不急着在这一层强行归一
- 示例：
  - **2023年销售总金额是多少？**
  - **2023年销售总额是多少？**
  - 这三种问法虽然语序和关键词不同，但都应先被收敛到同一个规范问题，例如“2023年销售订单总金额是多少”
  - 类似地，**VIP客户** 和 **会员客户** 如果在当前业务里本来指向同一类对象，也应在这一层被归并，而不是留到后面碰运气匹配
- 阶段结果：
  - 语序变化不再直接影响案例命中
  - 同义表达被尽量拉回到稳定说法
  - 系统关注的重点从“用户原句长什么样”变成“用户到底在问什么”

## 第三步：问题补全

第三步解决的是口语化问题，也就是缩写、省略、非正式表达和隐式语义补全。

走到这一步时，系统已经知道问题里哪些是参数，也已经尽量把词面表达归一了，但仍然会遇到一类更难的问题：用户根本没有把问题说完整。仅靠字面匹配，系统很难稳定地把它们拉回到正式案例上。

- 典型问题：
  - 时间被缩写，例如 23年 、 24年
  - 指标被口语化表达替代，例如 卖了多少钱
  - 主语、范围或口径被省略，只留下零散片段
  - 同一句话可能存在多种合理解释
- 解决方式：
  - 在前两步结果的基础上做受控补全，只补那些有明确依据的内容
  - 依据既包括已经识别出的参数，也包括已经归一的词组、业务知识和案例覆盖范围
  - 对能稳定还原的缩写、省略和口头说法，统一补成规范问题
  - 对仍然存在多种解释的表达，不强行补全，而是保留为待确认状态
- 示例：
  - 帮我查一下23年一共卖了多少钱？
  - 这句话可以先把 23年 还原为 2023年 ，再把 卖了多少钱 收敛为 “销售总金额”
  - 如果当前业务上下文已经明确是订单场景，这句话可以进一步补成 “2023年销售订单总金额是多少”
  - VIP客户24年平均每单花多少钱
  - 这类问题中， 24年 、 每单花多少钱 都需要先被还原，再补成类似 “VIP客户2024年平均客单价是多少” 的规范表达
  - 但像 今年四公司情况如何 这种范围过大的表达，就不应被硬补成某一个具体案例
- 阶段结果：
  - 常见口语问题可以进入正式比较链路
  - 缩写和省略不会再直接导致匹配失败
  - 歧义问题被留在可解释状态，而不是被过早误判

## 第四步：案例比较

前三步完成之后，系统手里应当已经有了一份相对稳定的问题表达。第四步才真正进入案例比较。

这一步要解决的，不再是“重新理解一遍用户问题”，而是“如何把已经整理过的问题，稳定地对齐到现有案例上”。如果前面三步做得足够扎实，那么到这里需要比较的其实已经不是原始问句，而是一个经过变量收敛、词组归一和问题补全之后的结果。

- 典型问题：
  - 当前流程仍然较依赖原始问句、摘要和逐案重解析
  - 匹配和提参没有稳定复用同一份问题理解
  - 精确匹配和相关匹配之间缺少清晰的分层判断
- 解决方式：
  - 案例比较直接使用前三步收敛后的结果，不再反复回到原始问题重新拆解

- 先比较问题主干是否一致，再比较参数范围是否兼容，最后再区分精确命中、相关命中和歧义状态
- 命中案例后，后续参数补齐、SQL 执行或者 API 执行继续复用同一份理解结果
- 示例：
  - 输入问题：帮我查一下23年一共卖了多少钱？
  - 经过前三步后，问题已经被收敛成类似“2023年销售订单总金额是多少”
  - 这时案例比较面对的就不再是原句，而是一个已经被整理过的问题表达
  - 如果案例库中存在“2023年销售订单总金额是多少”这一类规范案例，就应直接进入稳定比较，而不是再针对每个候选案例重新生成一份新的问题解释
- 阶段结果：
  - 案例比较建立在更稳定的输入上
  - 匹配与提参开始共享同一条问题理解链路
  - 后续 SQL 或 API 执行可以直接复用前面已经确认过的结果

### 5.3 API 场景的接入方式

除了 SQL 案例之外，系统后续还可能接入另一类能力：外部 API。它们和案例路径的目标是一致的，都是在用户问题命中某种稳定能力之后，直接复用现成结果，而不是重新生成一套新逻辑。但 API 场景和 SQL 案例有一个根本差异：

- SQL 案例通常能看到相对明确的计算逻辑
- API 往往只有参数说明、指标说明和返回结果说明
- API 本身不一定暴露内部如何计算，因此无法像 SQL 那样直接比较具体逻辑

这意味着，API 不能简单复用“参考 SQL 模板去比对”的思路，而要改成“围绕指标语义和参数契约去匹配”的方式。

#### API 接入时要解决的核心问题

- 系统需要判断这个 API 到底解决的是哪一类问题
- 系统需要判断用户问题里的参数，是否能稳定映射到 API 需要的输入参数
- 系统需要判断用户问题的范围，是否落在 API 的支持边界内

#### API 接入的整体思路

- 第一步仍然是参数识别和验证。
  - 这一点和 SQL 案例完全一致。
  - 如果用户问的是今年四公司产值最高的项目，系统仍然要先识别出今年、四公司、产值最高、项目这些关键部分，并尽量确认它们对应的时间、组织、指标对象和排序需求。
- 第二步仍然是词组归一化。
  - API 不会因为没有 SQL 就不需要归一。

- 如果用户说的是 **四公司今年最挣钱的项目**，而 API 文档里写的是“项目产值排名”，系统仍然要先把 **最挣钱** 这种表述收敛到和 API 指标说明一致的规范语义上。
- 第三步仍然是问题补全。
  - 如果用户只说 **查下四公司今年产值最高项目**，系统仍然需要把省略的表达补齐到可以稳定比较的程度。
  - 只有先把问题整理清楚，后面才能判断这个 API 是否真的能接。
- 第四步改成“问题和 API 能力说明的比较”。
  - 因为 API 没有显式 SQL 逻辑，所以这里比较的重点不再是“SQL 是否一致”，而是“用户要的指标语义、统计对象、必要参数和结果形态，是否与 API 的能力说明一致”。

## API 匹配时的判断依据

在 API 场景中，系统至少要基于以下几类信息来判断是否命中：

- 指标说明。
  - 这个 API 算的到底是什么，例如“项目产值排名”“组织月度销售额”“客户平均客单价”。
- 参数说明。
  - 这个 API 需要哪些参数，哪些是必填，哪些是可选，参数分别表示时间、组织、区域、项目还是其他业务对象。
- 支持范围说明。
  - 这个 API 支持按什么粒度查询，支持哪些筛选条件，是否支持排序、TopN、时间范围等能力。
- 返回结果说明。
  - 这个 API 返回的是单值、列表、排名结果，还是时间序列结果。
- 示例问法。
  - API 也应像案例一样补充一组典型问法，帮助系统把真实用户表达和 API 能力对齐。

也就是说，API 虽然没有 SQL，但不能只有一个接口名和几个参数。它仍然需要一份足够稳定的“能力说明”，否则系统无法可靠判断它适合回答什么问题。

## API 与案例的主要差异

- SQL 案例更容易从模板和字段关系中反推出稳定逻辑
- API 场景看不到内部逻辑，因此不能把“内部怎么算”作为匹配依据
- SQL 案例的精确匹配，更强调逻辑骨架是否一致
- API 场景的精确匹配，更强调指标语义是否一致、参数是否齐全、支持范围是否吻合

换句话说，API 的命中条件不应是“它内部可能也能算出这个结果”，而应是“它被定义出来就是为了解这类问题，并且当前问题所需参数都能稳定提供”。